

Phase 4: vLLM Runtime Enhancements

Overview

Phase 4 adds production-ready enhancements to the vLLM fallback implementation completed in Phases 1-3.

Status: OPTIONAL ENHANCEMENTS

The core vLLM fallback functionality is **complete and working**. These enhancements improve observability, configurability, and user experience but are not required for the system to function.

Enhancement 1: Fallback-Specific Metrics ✓ (Already Exists)

Current State

The LLM Router already has comprehensive metrics:

```
# Existing metrics in llm_router_service.py (lines 80-115)
PROVIDER_SELECTION_COUNTER # Tracks provider selections
PROVIDER_FALLBACK_COUNTER # Tracks fallback transitions ✓
PROVIDER_LATENCY_HISTOGRAM # Tracks provider latency
PROVIDER_FAILURE_COUNTER # Tracks provider failures
```

What's Already Working

- **PROVIDER_FALLBACK_COUNTER** tracks fallback transitions with labels:
 - **from_provider**: Original provider that failed
 - **to_provider**: Fallback provider used
 - **reason**: Why fallback occurred

PROF

Verification

```
# Check Prometheus metrics endpoint
curl http://localhost:8000/metrics | grep
kari_llm_provider_fallbacks_total
```

Status: ✓ **COMPLETE** - No action needed

Enhancement 2: Circuit Breaker Configuration ✓ (Already Exists)

Current State

The LLM Router has a sophisticated circuit breaker implementation:

```
# Existing configuration in llm_router_service.py (lines 230-249)
self.retry_attempts = 3
self.retry_initial_delay = 1.0
self.retry_backoff_factor = 2.0
self.retry_max_delay = 10.0
self.retry_jitter = 0.5

self.circuit_breaker_threshold = 3
self.circuit_breaker_timeout = 60.0

self.rate_limit_backoff = 15.0
self.latency_history_size = 20

self.default_rate_limit = {"max_requests": 30, "window_seconds": 60}
self.rate_limit_config: Dict[str, Dict[str, float]] = {
    "openai": {"max_requests": 60, "window_seconds": 60},
    "anthropic": {"max_requests": 30, "window_seconds": 60},
    "gemini": {"max_requests": 40, "window_seconds": 60},
    "deepseek": {"max_requests": 40, "window_seconds": 60},
}
```

What's Already Working

- **Circuit Breaker:** Opens after 3 consecutive failures, stays open for 60 seconds
- **Retry Logic:** 3 attempts with exponential backoff (1s → 2s → 4s, max 10s)
- **Rate Limiting:** Per-provider rate limits with cooldown periods
- **Health Tracking:** Tracks consecutive failures, response times, error types

Optional: Environment Variable Configuration

If you want to make these configurable via environment variables, add to `.env`:

```
# Circuit Breaker Configuration (Optional)
KAREN_CIRCUIT_BREAKER_THRESHOLD=3
KAREN_CIRCUIT_BREAKER_TIMEOUT=60
KAREN_RETRY_ATTEMPTS=3
KAREN_RETRY_INITIAL_DELAY=1.0
KAREN_RETRY_BACKOFF_FACTOR=2.0
KAREN_RETRY_MAX_DELAY=10.0
```

Then modify `llm_router_service.py` `__init__` to read from environment:

```
self.circuit_breaker_threshold =
int(os.getenv("KAREN_CIRCUIT_BREAKER_THRESHOLD", "3"))
self.circuit_breaker_timeout =
```

```
float(os.getenv("KAREN_CIRCUIT_BREAKER_TIMEOUT", "60.0"))
self.retry_attempts = int(os.getenv("KAREN_RETRY_ATTEMPTS", "3"))
# ... etc
```

Status: ✓ **COMPLETE** - Optionally add env var support

Enhancement 3: Provider Health Caching ✓ (Already Exists)

Current State

The LLM Router has comprehensive health tracking:

```
# Existing health tracking (lines 139-157)
@dataclass
class ProviderHealth:
    name: str
    is_healthy: bool
    last_check: float
    response_time: Optional[float] = None
    error_message: Optional[str] = None
    consecutive_failures: int = 0
    last_failure: Optional[float] = None
    circuit_open_until: float = 0.0
    rate_limited_until: float = 0.0
    requests_in_window: int = 0
    window_start: float = 0.0
    latency_samples: List[float] = field(default_factory=list)
    last_exception_type: Optional[str] = None
    total_requests: int = 0
```

—
PROF

What's Already Working

- Health checks cached for 5 minutes (`health_check_interval = 300`)
- Background health monitoring every 3 minutes (`background_health_interval = 180`)
- Circuit breaker state prevents unnecessary health checks
- Latency samples tracked for performance analysis

Status: ✓ **COMPLETE** - No action needed

Enhancement 4: UI Fallback Indicator 🚧 (Recommended)

Current State

The UI displays provider metadata from the backend, but doesn't have a visual indicator for fallback scenarios.

Recommended Addition

Add a visual indicator in the chat response card when fallback occurs.

File to Modify

src/ui_launchers/Karen-AI-Theme/src/components/chat/MessageCard.tsx (or equivalent)

Implementation

```
// Add to message metadata display
interface MessageMetadata {
  provider?: string;
  actual_provider?: string;
  requested_provider?: string;
  is_degraded?: boolean;
  fallback_level?: number;
  fallback_chain?: string[];
  response_source?: string;
}

// Visual indicator component
function FallbackIndicator({ metadata }: { metadata: MessageMetadata }) {
  if (!metadata.is_degraded || metadata.response_source ===
'emergency_static') {
    return null;
  }

  const usedFallback = metadata.actual_provider !==
metadata.requested_provider;

  if (!usedFallback) {
    return null;
  }

  return (
    <div className="fallback-indicator" title={`Fallback:
${metadata.requested_provider} → ${metadata.actual_provider}`}>
      <svg className="fallback-icon" /* ... icon SVG ... */>
        { /* Shield or routing icon */ }
      </svg>
      <span className="fallback-text">
        Fallback: {metadata.requested_provider} →
{metadata.actual_provider}
      </span>
    </div>
  );
}

// Add to message card
```

```
<MessageCard>
  {/* ... existing content ... */}
  <FallbackIndicator metadata={message.metadata} />
</MessageCard>
```

Styling

```
.fallback-indicator {
  display: inline-flex;
  align-items: center;
  gap: 0.5rem;
  padding: 0.25rem 0.75rem;
  background: rgba(255, 193, 7, 0.1);
  border: 1px solid rgba(255, 193, 7, 0.3);
  border-radius: 0.375rem;
  font-size: 0.875rem;
  color: #f59e0b;
  margin-top: 0.5rem;
}

.fallback-icon {
  width: 1rem;
  height: 1rem;
}

.fallback-text {
  font-weight: 500;
}
```

Status: 🛑 **RECOMMENDED** - Optional UI enhancement

PROF

Enhancement 5: Fallback Configuration File 🛑 (Optional)

Purpose

Centralize fallback chain configuration for easy tuning without code changes.

Implementation

Create Configuration File

config/fallback_config.yml:

```
# Fallback Chain Configuration
fallback:
  enabled: true
```

```
# Fallback chain order (tried in sequence)
chain:
  - provider: builtin_vllm
    priority: 1
    enabled: true
    timeout_seconds: 30

  - provider: builtin_transformers
    priority: 2
    enabled: true
    timeout_seconds: 45

  - provider: emergency
    priority: 3
    enabled: true
    timeout_seconds: 1

# Circuit breaker settings
circuit_breaker:
  threshold: 3 # failures before opening
  timeout_seconds: 60 # how long to keep open
  half_open_requests: 1 # requests to try in half-open state

# Retry settings
retry:
  max_attempts: 3
  initial_delay_seconds: 1.0
  backoff_factor: 2.0
  max_delay_seconds: 10.0
  jitter: 0.5

# Health check settings
health:
  check_interval_seconds: 300 # 5 minutes
  background_interval_seconds: 180 # 3 minutes
  latency_history_size: 20

# Rate limiting
rate_limits:
  default:
    max_requests: 30
    window_seconds: 60

  providers:
    openai:
      max_requests: 60
      window_seconds: 60
    anthropic:
      max_requests: 30
      window_seconds: 60
    gemini:
```

```
max_requests: 40
window_seconds: 60
```

Load Configuration

Modify `llm_router_service.py` to load from config:

```
import yaml
from pathlib import Path

def load_fallback_config():
    """Load fallback configuration from YAML file."""
    config_path = Path("config/fallback_config.yml")
    if not config_path.exists():
        return {}

    with open(config_path) as f:
        return yaml.safe_load(f)

class LLMRouter:
    def __init__(self, registry: Optional[Any] = None):
        # ... existing code ...

        # Load configuration
        config = load_fallback_config()
        fallback_config = config.get("fallback", {})

        # Apply configuration
        cb_config = fallback_config.get("circuit_breaker", {})
        self.circuit_breaker_threshold = cb_config.get("threshold", 3)
        self.circuit_breaker_timeout = cb_config.get("timeout_seconds",
60.0)

        retry_config = fallback_config.get("retry", {})
        self.retry_attempts = retry_config.get("max_attempts", 3)
        self.retry_initial_delay =
        retry_config.get("initial_delay_seconds", 1.0)
        # ... etc
```

PROF

Status: 🐞 **OPTIONAL** - Nice to have for production tuning

Enhancement 6: Grafana Dashboard 📊 (Optional)

Purpose

Visualize fallback metrics, circuit breaker state, and provider health.

Dashboard Panels

1. Fallback Rate

- Query: `rate(kari_llm_provider_fallbacks_total[5m])`
- Type: Graph
- Shows fallback frequency over time

2. Fallback Chain Heatmap

- Query: `kari_llm_provider_fallbacks_total`
- Type: Heatmap
- Shows which fallback paths are most common

3. Provider Health Status

- Query: Custom from `/api/health/providers/all`
- Type: Stat panels
- Shows current health of each provider

4. Circuit Breaker State

- Query: Custom metric or log-based
- Type: State timeline
- Shows when circuit breakers open/close

5. Provider Latency

- Query: `kari_llm_provider_latency_seconds`
- Type: Graph
- Shows response times by provider

Dashboard JSON

Create `docker/grafana/dashboards/vllm-fallback-dashboard.json`:

```
{
  "dashboard": {
    "title": "Karen vLLM Fallback Monitoring",
    "panels": [
      {
        "title": "Fallback Rate",
        "targets": [{
          "expr": "rate(kari_llm_provider_fallbacks_total[5m])"
        }]
      }
      // ... more panels
    ]
  }
}
```

Status:  **OPTIONAL** - For production monitoring

Summary

✓ Already Complete (No Action Needed)

1. **Fallback Metrics** - `PROVIDER_FALLBACK_COUNTER` tracks all fallback transitions
2. **Circuit Breaker** - Sophisticated implementation with configurable thresholds
3. **Health Caching** - 5-minute cache with background monitoring
4. **Retry Logic** - Exponential backoff with jitter

🔧 Recommended (Optional)

1. **UI Fallback Indicator** - Visual feedback when fallback occurs
2. **Environment Variable Config** - Make circuit breaker tunable via `.env`

📋 Nice to Have (Future)

1. **YAML Configuration File** - Centralized fallback configuration
2. **Grafana Dashboard** - Visual monitoring of fallback behavior

Implementation Priority

High Priority (Do Now)

- ✓ **None** - Core functionality is complete

Medium Priority (Consider)

- 🔧 **UI Fallback Indicator** - Improves user experience
- 🔧 **Environment Variable Config** - Easier production tuning

Low Priority (Future)

- 📋 **YAML Config File** - Nice for complex deployments
- 📋 **Grafana Dashboard** - Professional monitoring

PROF

Testing Phase 4 Enhancements

Test Fallback Metrics

```
# Generate fallback event (disable Gemini)
curl -X POST http://localhost:8000/api/chat/chat \
  -H "Content-Type: application/json" \
  -d '{"message":"test","provider":"gemini"}'

# Check metrics
curl http://localhost:8000/metrics | grep
kari_llm_provider_fallbacks_total
```

Test Circuit Breaker

```
# Trigger multiple failures
for i in {1..5}; do
  curl -X POST http://localhost:8000/api/chat/chat \
    -H "Content-Type: application/json" \
    -d '{"message":"test","provider":"invalid_provider"}'
done

# Check circuit breaker opened
curl http://localhost:8000/api/health/providers/all | jq
'.providers.invalid_provider'
```

Test Health Caching

```
# First call (cache miss)
time curl http://localhost:8000/api/health/providers/vllm

# Second call (cache hit, should be faster)
time curl http://localhost:8000/api/health/providers/vllm
```

Conclusion

Phase 4 Status: OPTIONAL ENHANCEMENTS

The vLLM fallback system is **production-ready** as-is. All critical functionality exists:

- ✓ Fallback chain works (Phases 1-3)
- ✓ Metrics track fallback events
- ✓ Circuit breaker prevents cascade failures
- ✓ Health monitoring prevents bad provider selection
- ✓ Retry logic handles transient failures

Phase 4 enhancements improve **observability** and **configurability** but are not required for the system to function correctly.

Recommendation: Deploy current implementation, add Phase 4 enhancements based on production needs.